Fiche Résumé : Vers un code Immuable

Partie Roman

L'essentiel en une phrase

Un objet immuable a un état figé dès sa création : toute « modification » produit une nouvelle instance au lieu de changer l'objet existant.

1. Définition

	Objet mutable	Objet immuable
État	Peut changer après la création (setters, méthodes modificatrices)	Figé dès la création
Modification	Change l'objet existant	Renvoie une nouvelle instance
Exemple	Panier avec `setTotal`	`PanierImmuable.avecReduction(20)`

Même besoin, deux comportements. Avec un objet mutable, la référence partagée est altérée :

```
Panier commande = new Panier(50);
Panier promo = commande; // même référence
promo.setTotal(30);
commande.getTotal(); // 30 au lieu de 50 !
```

Avec un objet immuable, l'original reste intact :

```
PanierImmuable commande = new PanierImmuable(50);
PanierImmuable promo = commande.avecReduction(20);
commande.getTotal(); // 50 (intact)
promo.getTotal(); // 30 (nouvelle instance)
```

2. Pourquoi s'y intéresser ?

Trois questions que tout développeur finit par se poser :

- Qui a modifié mon objet ?
- Pourquoi mon résultat change-t-il selon l'ordre d'exécution ?
- Pourquoi mon objet a-t-il disparu de ma `HashMap` ?

Les trois bugs ont la même cause : un état qui change sans qu'on le voie.

3. Avantages

- **Prévisibilité :** aucune méthode ne peut modifier l'objet en douce.
- **Thread-safety native :** partageable entre threads sans synchronized, à condition que l'objet soit entièrement immuable.
- **Fiabilité dans les collections :** le hashCode ne change plus, donc l'objet reste retrouvable dans un HashSet ou une HashMap.
- **Testabilité :** état stable, tests déterministes.

4. Limites et compromis

- **Coût d'instanciation** : chaque « modification » crée un objet. C'est négligeable le plus souvent, mais sensible dans des boucles serrées (traitement d'images, calculs matriciels).
- **Verbosité en Java classique** : constructeur complet, copies défensives, `equals`, `hashCode` et `toString` à écrire à la main.
- **Pas adapté à tout** : un objet dont l'état évolue naturellement dans le métier (un compte, un panier en cours) se prête mal à l'immuabilité.

5. La recette pour une classe immuable en Java

Règle	Contenu	Pourquoi
1	Classe `finale`	Interdit l'héritage : aucune sous-classe ne peut ajouter de setter

2	Attributs `private final`	Affectés une seule fois, dans le constructeur
3	Aucun setter	Pour « modifier », on renvoie une nouvelle instance (`avecPrix(...)`)
4	Copie défensive à l'entrée	L'appelant ne garde pas de référence vers notre état
5	Copie défensive à la sortie	L'accesseur ne laisse pas fuir un objet mutable
6	`equals, hashCode, toString`	Égalité par valeurs, cohérente avec le `hashCode`

La classe `Article` qui applique les six règles :

```
import java.util.Date;
import java.util.Objects;

public final class Article { // Règle 1
 private final String nom; // Règle 2
 private final double prix;
 private final Date dateAjout;

public Article(String nom, double prix, Date dateAjout) {
 this.nom = nom;
 this.prix = prix;
 this.dateAjout = new Date(dateAjout.getTime()); // Règle 4
}

public String getNom() { return nom; } // Règle 3 : pas de setter
 public double getPrix() { return prix; }
 public Date getDateAjout() {
 return new Date(dateAjout.getTime()); // Règle 5
}

public Article avecPrix(double nouveauPrix) { // « modifier » = nouvelle
instance
 return new Article(nom, nouveauPrix, dateAjout);
}

@Override // Règle 6
 public boolean equals(Object o) {
 if (this == o) return true;
```

```
 if (o == null || getClass() != o.getClass()) return false;
 Article other = (Article) o;
 return Double.compare(prix, other.prix) == 0
 && nom.equals(other.nom)
 && dateAjout.equals(other.dateAjout);
}

@Override
 public int hashCode() { return Objects.hash(nom, prix, dateAjout); }

@Override
 public String toString() {
 return "Article[nom=" + nom + ", prix=" + prix + ", dateAjout=" +
dateAjout + "]";
 }
}
```

Les pièges à connaître

- `Final class` ≠ classe immuable : une classe `final` qui possède un setter reste modifiable. `final` sur la classe interdit seulement l'héritage.
- `final` sur un attribut ≠ immuable : il interdit de réaffecter la référence, pas de modifier l'objet pointé (exemple : Date).
- Les copies défensives sont inutiles pour `String` et `double`, qui sont déjà immuables.

6. Le cas critique des collections

Le mythe de final. `final` bloque la réassignation de la variable, pas la modification du contenu de la liste :

```
      final List<String> articles = new ArrayList<>();
      articles.add("Livre"); // tout à fait permis !
```

La fuite de référence. Une classe `final` avec un attribut `private final List` n'est pas immuable : le constructeur et l'accesseur laissent l'extérieur garder une référence vers la liste interne.

```
public final class Commande {
      private final List<String> articles;

      public Commande(List<String> articles) { this.articles = articles; }
      public List<String> getArticles() { return articles; }
}

List<String> liste = new ArrayList<>(List.of("Livre"));
```

```
Commande c = new Commande(liste);

liste.add("Stylo"); // fuite à l'entrée : la commande a changé
c.getArticles().clear(); // fuite à la sortie : la commande est vidée
```

Une collection standard n'est pas immuable parce que son API expose des méthodes de mutation (`add`, `remove`, `clear`…), pas parce que son contenu varie.

7. À retenir

- Immuable = état figé à la création, « modifier » = créer une nouvelle instance.
- Recette : classe `final`, attributs `private final`, pas de setter, copies défensives à l'entrée et à la sortie, `equals`/`hashCode`/`toString`.
- `final` ne suffit ni sur la classe, ni sur un attribut, ni sur une collection.
- Une collection immuable ne garantit rien sur ses éléments : viser l'immuabilité profonde.
- Le prix à payer : verbosité et création d'objets. Java moderne (records, `List.copyOf`) réduit ce coût : voir les autres parties de l'exposé.

Partie Martial

1. Immutabilité

Un objet est **immuable** lorsque son état ne peut pas être modifié après sa création.

- `final class` empêche l'héritage, mais ne rend pas les objets immuables.
- Un champ `final` ne peut pas être réaffecté, mais l'objet qu'il référence peut rester mutable.
- Supprimer les setters et protéger les données contribue à l'immutabilité.

2. Les `record` en Java

Un `record` est une forme concise de classe adaptée à la représentation de données.

```
public record BoardMoveHint(int x, int y, int score) {}
```

Java génère notamment le constructeur, les accesseurs (`x()`, `y()`, `score()`), `equals()`, `hashCode()` et `toString()`.

- Les composants d'un `record` sont `final`.
- Un constructeur compact permet de valider les valeurs à la création.
- Un `record` n'est pas automatiquement profondément immuable si ses composants référencent des objets mutables.
- Les `record` sont définitifs depuis Java 16.

3. Validation des données

`Objects.requireNonNull(value)` rejette une référence `null` en levant une `NullPointerException`. Une chaîne vide (`""`) reste acceptée.

Le constructeur compact permet de vérifier les invariants dès la création de l'objet.

À retenir : la validation ne rend pas un objet immuable à elle seule.

4. Collections Java

- `final List<String>` empêche de réaffecter la variable, mais pas de modifier le contenu de la liste.
- `List.of(...)` crée une liste non modifiable.
- `List.copyOf(collection)` renvoie une copie structurelle non modifiable. La collection source reste modifiable.

Une tentative de modification d'une liste non modifiable provoque une `UnsupportedOperationException`.

5. Immutabilité superficielle et profonde

- **Immutabilité superficielle** (*shallow immutability*) : la structure de la collection est protégée, mais les objets qu'elle contient peuvent rester mutables.
- **Immutabilité profonde** (*deep immutability*) : l'état des objets accessibles depuis la collection est également protégé.

`List.copyOf()` ne clone pas les objets contenus.

6. Collections persistantes : Vavr

Vavr fournit des collections immuables persistantes.

Une opération comme `append()` renvoie une nouvelle liste sans modifier l'ancienne. Les versions précédentes restent disponibles.

L’immuabilité de la collection ne garantit pas automatiquement celle de ses éléments.

À retenir

1. `final class` ne signifie pas classe immuable.
2. Les `record` réduisent le code répétitif et conviennent aux objets de données.
3. Le constructeur compact permet de valider les invariants à la création.
4. Une liste non modifiable ne rend pas automatiquement ses éléments immuables.
5. Pour viser une immutabilité profonde, il faut examiner les objets référencés.

Partie Pierre

1. Le Domain-Driven Design (DDD)

Le **Domain-Driven Design** est une approche de conception qui consiste à organiser le logiciel autour du métier. Il permet de représenter les concepts métier directement dans le code. On distingue deux familles de patterns : les **patterns stratégiques**, qui organisent le domaine, et les **patterns tactiques**, qui servent à modéliser les objets métier. Nous nous intéressons ici aux patterns Entity et Value Object.

2. Le pattern Entity

Une **Entity** est un objet métier qui possède une identité unique. Ses attributs peuvent évoluer au cours du temps, mais son identité reste la même. Par exemple, un **Client** peut changer de nom ou d’adresse email en conservant son identifiant.

En Java, l’identifiant est déclaré **final**, tandis que les autres attributs peuvent évoluer grâce à des méthodes métier. Les méthodes **equals()** et **hashCode()** sont redéfinies pour comparer les Entities à partir de leur identité.

3. Le pattern Value Object

Un **Value Object** représente une valeur métier sans identité propre. Deux Value Objects ayant les mêmes valeurs sont considérés comme égaux. Il est

immuable : pour représenter une nouvelle valeur, on crée une nouvelle instance.

En Java classique, le support utilise une classe **final**, des attributs **private final**, aucun setter, des validations et les méthodes **equals()** et **hashCode()**. Le **record** simplifie ensuite ce code en générant automatiquement les accesseurs et les méthodes de comparaison.

L'exemple Billet montre que deux objets représentant **20 EUR** sont égaux, même lorsqu'ils sont créés séparément. **En résumé** : l'Entity est définie par son identité, tandis que le Value Object est défini par ses valeurs.